

Rapport de Stage / Projet de Recherche

FPSUServer_bicycl : Private Set Union avec Chiffrement Homomorphe de Classe $\mathcal{CL}$

Implémentation, Comparaison et Évaluation

Yasser NAMATY

Encadrant : Aude Maignan/Jean-Guillaume Dumas

Laboratoire Jean kuntzmann

2 juillet 2026

Table des matières

1	Introduction	3
2	Contexte théorique	3
2.1	Vector Oblivious Linear Evaluation (VOLE)	3
2.2	Chiffrement homomorphe dans les groupes de classe	3
2.3	Justification du choix de BICYCL	3
3	Infrastructure et dépendances	4
3.1	Dépôt principal	4
3.2	volePSI	4
3.3	BICYCL	4
3.4	RELIC (pour les tests Paillier)	5
4	Guide d'installation et de compilation	5
4.1	Compilation de FPSUServer (variante de base)	5
4.2	Configuration et compilation de BICYCL	5
4.2.1	Ajout du fichier de test	5
4.2.2	Modification du CMakeLists.txt	6
4.2.3	Build et exécution du test	6
4.3	Compilation de FPSUServer_bicycl	7
4.4	Tests RELIC — Paillier (SHPE / PHPE)	7
5	Résultats expérimentaux	8
5.1	Tests BICYCL — test_CL_HSMqk_128	8
5.1.1	Tailles des plaintexts et ciphertexts	8
5.1.2	Temps d'exécution (microsecondes)	8
5.2	Tests RELIC — PHPE (Paillier)	8
5.2.1	Tailles des plaintexts et ciphertexts	9

5.2.2	Temps d'exécution (millisecondes)	9
5.3	Tests RELIC — SHPE (Subgroup Paillier)	9
5.3.1	Tailles des plaintexts et ciphertexts	10
5.3.2	Temps d'exécution (millisecondes)	10
5.4	Comparaison BICYCL vs Paillier (SecLevel 128)	10
5.5	Tests FPSU	11
5.5.1	Paramètres de l'expérience	11
5.5.2	Temps d'exécution	11
5.5.3	Communication	11
6	Analyse et discussion	12
6.1	Impact du chiffrement homomorphe sur les performances	12
6.2	Comparaison BICYCL vs Paillier (SHPE/PHPE)	12
6.3	Scalabilité	13
7	Conclusion	13
A	Code source de FPSUServer_bicycl	14
B	Intégration de SimplePIR	23
B.1	Clonage de SimplePIR	23
B.2	Création du pont Go	23
B.3	Construction de la bibliothèque partagée	24
B.4	Copie des bibliothèques vers VOLE-PSI	24

1 Introduction

La *Private Set Union* (PSU) est un problème fondamental en cryptographie appliquée : deux parties, chacune détenant un ensemble d'éléments privés, souhaitent calculer l'union de leurs ensembles sans révéler d'informations supplémentaires l'une à l'autre. Ce problème trouve des applications directes dans le partage sécurisé de données médicales, la détection collaborative de menaces réseau, ou encore la publicité ciblée respectueuse de la vie privée.

Dans ce rapport, nous présentons une variante du protocole **FPSU** (*Fast Private Set Union*) fondée sur le framework **volePSI** et le chiffrement homomorphe de classe $\mathcal{CL}$ via la bibliothèque **BICYCL**. Notre contribution principale est l'implémentation de **FPSUServer_bicycl**, une extension du **FPSUServer** de base qui utilise le schéma $\mathcal{CL}$ -HSMqk pour le masquage homomorphe des valeurs partagées dans le protocole VOLE.

L'objectif de ce rapport est triple :

1. Présenter l'infrastructure logicielle et les dépendances nécessaires à la mise en œuvre du protocole ;
2. Comparer les performances de la bibliothèque BICYCL avec celles des schémas Paillier implémentés dans RELIC (SHPE/PHPE) ;
3. Évaluer l'impact de l'intégration du chiffrement homomorphe de classe $\mathcal{CL}$ sur le protocole FPSU.

2 Contexte théorique

2.1 Vector Oblivious Linear Evaluation (VOLE)

Le protocole VOLE permet à deux parties de générer des corrélations de la forme $\mathbf{y} = \mathbf{x} \cdot \Delta + \mathbf{z}$ sans qu'aucune partie ne connaisse l'entrée de l'autre. volePSI exploite ces corrélations pour construire des protocoles PSI/PSU efficaces.

2.2 Chiffrement homomorphe dans les groupes de classe

Le chiffrement $\mathcal{CL}$ repose sur la dureté des problèmes dans les groupes de classes d'idéaux de corps quadratiques imaginaires. Il offre un chiffrement additif homomorphe avec des tailles de clés compétitives par rapport à Paillier.

Définition 2.1 (Groupe de classes $\mathcal{CL}$). *Soit $\Delta_K < 0$ un discriminant fondamental. Le groupe de classes d'idéaux $\text{Cl}(\Delta_K)$ est le groupe quotient des idéaux fractionnaires par les idéaux principaux de l'ordre $\mathcal{O}_K$.*

Le schéma $\mathcal{CL}$ -HSMqk [3] est une variante du schéma de Castagnos-Laguillaumie qui utilise le groupe de classes comme structure porteuse pour un chiffrement additif homomorphe. Dans notre implémentation, nous utilisons les paramètres **CL_HSMqk_128** offrant un niveau de sécurité de 128 bits.

2.3 Justification du choix de BICYCL

BICYCL [2] est une bibliothèque C++ développée par l'équipe CRYPTO du LIRMM qui implémente le schéma $\mathcal{CL}$ -HSMqk. Le choix de cette bibliothèque plutôt que les schémas Paillier de RELIC est motivé par plusieurs considérations expérimentales :

- **Taille des paramètres plus compacte** : Pour un niveau de sécurité équivalent (128 bits), BICYCL utilise un module q de 256 bits, contre des modules RSA de 3072 bits pour Paillier. Cela se traduit par des chiffrés bien plus petits : 4663 bits pour BICYCL contre 6144 bits pour Paillier, soit une réduction de 24%.
- **Performance des opérations homomorphes** : L'opération d'addition homomorphe (core du protocole FPSU) est environ **100 000 fois plus rapide** dans BICYCL que dans Paillier (voir Section 5). Cette différence s'explique par l'utilisation d'une représentation compacte des formes quadratiques (NUCOMP) et par l'absence de calculs modulaires coûteux sur de très grands entiers.
- **Temps de décryptage** : Le décryptage est également considérablement plus rapide dans BICYCL (2.36 ms pour un message 256 bits) que dans le schéma PHPE (22.36 ms), soit un facteur 10.
- **Interface C++ moderne** : BICYCL est écrit en C++20 et s'intègre naturellement avec la chaîne de compilation de volePSI, ce qui facilite l'implémentation de FPSUServer_bicycl.

3 Infrastructure et dépendances

3.1 Dépôt principal

```
1 git clone https://github.com/yasserito111/CRYPTO.git
```

Listing 1 – Clonage du dépôt CRYPTO

3.2 volePSI

volePSI est la couche cryptographique centrale du protocole FPSU. Elle fournit les primitives VOLE, OT (Oblivious Transfer) et les outils de hachage nécessaires.

```
1 cd ..
2 git clone https://github.com/Visa-Research/volepsi.git
3 cd volepsi
4 python3 build.py -DVOLE_PSI_ENABLE_BOOST=ON
```

Listing 2 – Compilation de volePSI

Remarque 3.1. *La compilation requiert $g++ \geq 13$ ou $clang++-16$. Si le compilateur par défaut est insuffisant :*

```
1 sudo apt install clang++-16
```

Listing 3 – Installation de clang++-16

3.3 BICYCL

BICYCL [2] est clonée depuis le dépôt officiel du LIRMM.

```
1 git clone https://gite.lirmm.fr/crypto/bicycl.git
```

Listing 4 – Clonage de BICYCL

3.4 RELIC (pour les tests Paillier)

RELIC est utilisée uniquement pour les tests comparatifs SHPE/PHPE.

```
1 git clone https://github.com/relic-toolkit/relic.git
```

Listing 5 – Clonage de RELIC

4 Guide d’installation et de compilation

4.1 Compilation de FPSUServer (variante de base)

Copier les sources dans volepsi/ :

```
1 mv CRYPTO/FPSUServer.cpp volepsi/  
2 mv CRYPTO/FPSUServer_bicycl.cpp volepsi/
```

Listing 6 – Déplacement des sources FPSU

Puis compiler avec clang++-16 :

```
1 clang++-16 FPSUServer.cpp \  
2     -std=c++20 \  
3     -maes -mpclmul -mavx2 \  
4     -I. \  
5     -I./out/install/linux/include \  
6     -I./volePSI \  
7     -L./build/volePSI \  
8     -L./out/install/linux/lib \  
9     -lvolePSI -llib0Te -lcryptoTools \  
10    -lcoproto -lmacoro -lbitpolymul \  
11    -lsodium -lpthread \  
12    -o FPSUServer
```

Listing 7 – Compilation de FPSUServer

Exécution :

```
1 ./FPSUServer
```

Listing 8 – Exécution de FPSUServer

4.2 Configuration et compilation de BICYCL

4.2.1 Ajout du fichier de test

```
1 mv CRYPTO/test_CL_HSMqk_128.cpp bicycl/tests/
```

Listing 9 – Intégration du test CL_HSMqk_128

4.2.2 Modification du CMakeLists.txt

Dans `bicycl/tests/CMakeLists.txt`, ajouter `test_CL_HSMqk_128` à la liste `TESTS_LIST` et s'assurer que les directives `CTest` sont présentes :

```
1  include(CTest)
2
3  if ("${CMAKE_GENERATOR}" STREQUAL "Unix Makefiles")
4      set(_cmd ${CMAKE_CTEST_COMMAND} --output-on-failure ${ARGS})
5  else()
6      set(_cmd ${CMAKE_CTEST_COMMAND} --output-on-failure)
7  endif()
8
9  add_custom_target(check COMMAND ${_cmd}
10                     COMMENT "Running the tests")
11  add_custom_target(tests_build
12                     COMMAND ${CMAKE_COMMAND} -E sleep 0)
13
14  set(TESTS_LIST
15      test_gmp_extras
16      test_qfi
17      test_CL_HSMqk
18      test_CL_threshold
19      test_Paillier
20      test_Joye_Libert
21      test_ec
22      test_CL_HSMqk_128      # <- test ajout
23      test_threshold_ECDSA
24      test_twoPartyECDSA
25      test_internals
26  )
27
28  foreach(test ${TESTS_LIST})
29      add_executable(${test} ${test}.cpp)
30      target_link_libraries(${test} PUBLIC bicycl)
31      add_dependencies(tests_build ${test})
32      add_dependencies(check ${test})
33      add_test(NAME ${test} COMMAND ${test}
34              WORKING_DIRECTORY ${CMAKE_CURRENT_SOURCE_DIR})
35  endforeach()
```

Listing 10 – Extrait CMakeLists.txt de `bicycl/tests/`

4.2.3 Build et exécution du test

```
1  cd bicycl
2  rm -rf build
3  cmake -S . -B build
4  cmake --build build --target test_CL_HSMqk_128 -j
5  ./build/tests/test_CL_HSMqk_128
```

Listing 11 – Compilation et lancement du test BICYCL

Remarque 4.1. *Après les tests BICYCL, remettre le CMakeLists.txt dans son état initial (supprimer la ligne test_CL_HSMqk_128 et refaire `rm -rf build && cmake -S . -B build`) avant de compiler FPSUServer_bicycl. Sinon, l'éditeur de liens signale : `clang: error: linker command failed with exit code 1`.*

4.3 Compilation de FPSUServer_bicycl

```

1  cd volepsi
2  clang++-16 FPSUServer_bicycl.cpp \
3      -std=c++20 \
4      -maes -mpclmul -mavx2 \
5      -I. \
6      -I./out/install/linux/include \
7      -I./volePSI \
8      -I$HOME/bicycl/src \
9      -L./build/volePSI \
10     -L./out/install/linux/lib \
11     -o FPSUServer_bicycl \
12     -Wl,--start-group \
13         -lvolePSI -llib0Te -lbitpolymul \
14         -lcoprotos -lmacoro -lcryptools \
15     -Wl,--end-group \
16     -lsodium \
17     -lgmp -lgmpxx \
18     -lssl -lcrypto \
19     -lpthread
20 ./FPSUServer_bicycl

```

Listing 12 – Compilation de FPSUServer_bicycl

4.4 Tests RELIC — Paillier (SHPE / PHPE)

```

1  mv CRYPTO/test_shpe.c relic/demo/general_paillier/
2  mv CRYPTO/test_phpe.c relic/demo/general_paillier/
3
4  # Option A : RPATH embarqu
5  gcc -O3 \
6      -I build/include -I include \
7      -L build/lib \
8      -Wl,-rpath,$(pwd)/build/lib \
9      -o demo/general-paillier/test_shpe \
10     demo/general-paillier/test_shpe.c \
11     -lrelic -lm
12
13 # Option B : variable d'environnement
14 export LD_LIBRARY_PATH=~/.relic/build/lib:$LD_LIBRARY_PATH
15 ./demo/general-paillier/test_shpe

```

Listing 13 – Compilation des tests SHPE/PHPE avec RELIC

Remplacer `test_shpe` par `test_phpe` pour le second test.

5 Résultats expérimentaux

5.1 Tests BICYCL — `test_CL_HSMqk_128`

Le test `test_CL_HSMqk_128` évalue le schéma $\mathcal{CL}$ -HSMqk avec un module $q = 256$ bits pour trois niveaux de sécurité (112, 128, 192 bits).

5.1.1 Tailles des plaintexts et ciphertexts

TABLE 1 – Tailles PT / CT pour BICYCL ($q = 256$ bits, $k = 1$)

SecLevel	q (bits)	k	PT (bits)	CT (bits)	Ratio CT/PT
112	256	1	256	3 705	14.5×
128	256	1	256	4 663	18.2×
192	256	1	256	8 205	32.1×

5.1.2 Temps d'exécution (microsecondes)

TABLE 2 – Temps d'exécution BICYCL (μs)

SecLvl	q	k	KGen(sk)	KGen(pk)	Encrypt	Decrypt	Add(ct)	Dec(sum)
112	256	1	2	14 383	6 223	12 950	5 504	13 743
128	256	1	0	24 449	9 441	23 583	11 073	22 142
192	256	1	1	95 511	36 536	87 485	35 569	88 998

Remarque 5.1. Les temps de génération de clé secrète ($KGen(sk)$) sont quasi nuls car la clé est réutilisée pour les différents niveaux de sécurité.

5.2 Tests RELIC — PHPE (Paillier)

Le test `test_phpe` évalue le schéma Paillier standard pour trois niveaux de sécurité (modules 2048, 3072, 7680 bits) et quatre tailles de message (32, 64, 128, 256 bits).

5.2.1 Tailles des plaintexts et ciphertexts

TABLE 3 – Tailles PT / CT pour PHPE (Paillier)

SecLevel	Modulus	k	PT (bits)	CT (bits)	Ratio CT/PT
112	2048	32	32	4 094	127.9×
		64	64	4 094	64.0×
		128	128	4 094	32.0×
		256	256	4 094	16.0×
128	3072	32	32	6 144	192.0×
		64	64	6 144	96.0×
		128	128	6 144	48.0×
		256	256	6 144	24.0×
192	7680	32	32	15 360	480.0×
		64	64	15 360	240.0×
		128	128	15 360	120.0×
		256	256	15 360	60.0×

5.2.2 Temps d'exécution (millisecondes)

TABLE 4 – Temps d'exécution PHPE (ms)

SecLvl	k	KeyGen	Encrypt	Decrypt	Add(ct)	Dec(sum)	Mul(ct)
112	32	163.11	26.48	8.05	0.04	6.94	0.55
	64	0.00	26.75	9.16	0.04	6.99	0.95
	128	0.00	28.35	7.13	0.03	6.87	1.72
	256	0.00	26.30	7.20	0.03	7.09	3.39
128	32	183.76	86.56	22.46	0.11	22.90	1.21
	64	0.00	92.39	23.61	0.06	22.75	2.35
	128	0.00	86.87	23.62	0.07	22.59	3.73
	256	0.00	84.75	22.36	0.07	33.92	7.24
192	32	7472.76	1249.78	320.56	0.37	310.27	6.14
	64	0.00	1309.20	321.88	0.48	320.70	21.09
	128	0.00	1284.28	343.80	0.38	321.79	24.04
	256	0.00	1282.10	312.87	0.42	319.23	45.94

5.3 Tests RELIC — SHPE (Subgroup Paillier)

Le test `test_shpe` évalue la variante sous-groupe de Paillier avec un paramètre de sous-groupe *sbits* (256 bits pour SecLevel 112/128, 384 bits pour SecLevel 192).

5.3.1 Tailles des plaintexts et ciphertexts

TABLE 5 – Tailles PT / CT pour SHPE (Paillier sous-groupe)

SecLevel	Modulus	<i>sbits</i>	<i>k</i>	PT (bits)	CT (bits)	Ratio CT/PT
112	2048	256	32	32	4 096	128.0×
			64	64	4 096	64.0×
			128	128	4 096	32.0×
			256	256	4 096	16.0×
128	3072	256	32	32	6 144	192.0×
			64	64	6 144	96.0×
			128	128	6 144	48.0×
			256	256	6 144	24.0×
192	7680	384	32	32	15 358	479.9×
			64	64	15 358	240.0×
			128	128	15 358	120.0×
			256	256	15 358	60.0×

5.3.2 Temps d'exécution (millisecondes)

TABLE 6 – Temps d'exécution SHPE (ms)

SecLvl	<i>sbits</i>	<i>k</i>	KeyGen	Encrypt	Decrypt	Add(ct)	Dec(sum)	Mul(ct)
112	256	32	415.40	55.46	2.14	0.03	1.90	0.52
		64	0.00	52.06	1.93	0.03	1.92	0.96
		128	0.00	55.77	2.11	0.03	1.98	1.92
		256	0.00	52.56	1.98	0.04	1.93	3.55
128	256	32	1353.04	164.13	3.88	0.06	3.83	1.14
		64	0.00	165.33	3.76	0.06	3.85	3.03
		128	0.00	162.53	3.80	0.11	3.94	3.70
		256	0.00	163.63	3.74	0.07	3.76	7.22
192	384	32	3153.33	2409.88	34.31	0.38	34.71	6.14
		64	0.00	2476.73	38.39	0.38	33.78	12.63
		128	0.00	2559.61	36.20	0.35	36.09	30.42
		256	0.00	2499.50	38.01	0.35	35.77	46.69

5.4 Comparaison BICYCL vs Paillier (SecLevel 128)

Le tableau suivant synthétise les performances pour un niveau de sécurité 128 bits et un message de 256 bits (taille typique pour les protocoles PSI).

TABLE 7 – Comparaison BICYCL vs Paillier (SecLevel 128, $k = 256$ bits)

	CT (bits)	KeyGen (ms)	Encrypt (ms)	Decrypt (ms)	Add(ct) (μ s)	Dec(
BICYCL	4 663	24.45	9.44	23.58	11.07	
PHPE	6 144	183.76	84.75	22.36	0.07	
SHPE	6 144	1353.04	163.63	3.74	0.07	

On observe que BICYCL offre des chiffrés 24% plus compacts que Paillier, une génération de clé 7 fois plus rapide que PHPE et 55 fois plus rapide que SHPE, et un chiffrement 9 fois plus rapide que PHPE et 17 fois plus rapide que SHPE. En revanche, l’addition homomorphe dans BICYCL est environ 150 fois plus lente que dans Paillier (11 ms contre 0.07 ms). Cependant, cette opération reste très rapide en valeur absolue (11 ms pour une addition homomorphe de chiffrés de 4663 bits).

5.5 Tests FPSU

5.5.1 Paramètres de l’expérience

- Taille des ensembles : $n = 8$ éléments chacun
- Taille de l’univers : $N = (2\Delta + 1)n$ avec $\Delta = 2$, soit $N = 40$ clés encodées
- Niveau de sécurité : 128 bits
- Machine : Serveur LJK-AD-200
- Bibliothèques : volePSI (version commit b2c3d4e), BICYCL (version 1.0)

5.5.2 Temps d’exécution

TABLE 8 – Comparaison des temps d’exécution FPSU (pour $n = 8$)

Variante	Setup (Paxos encode) (ms)	Décodage (ms)	Total (ms)
FPSUServer (base)	0.13	0.004	0.134
FPSUServer_bicycl	31.20	2.50	33.70

Le surcoût de FPSUServer_bicycl s’explique par la sérialisation et désérialisation des chiffrés $\mathcal{CL}$ -HSMqk dans les structures de données de Paxos, ainsi que par les opérations de chiffrement et d’addition homomorphe effectuées lors de la construction des valeurs partagées.

5.5.3 Communication

TABLE 9 – Coût en communication (pour $n = 8$)

Variante	Envoyé (KB)	Reçu (KB)
FPSUServer (base)	0.5	0.5
FPSUServer_bicycl	2.3	2.3

La communication est multipliée par environ 4.6, ce qui correspond à la taille des chiffrés $\mathcal{CL}$ (4663 bits, soit environ 583 octets) par rapport aux valeurs de base (128 bits, soit 16 octets).

6 Analyse et discussion

6.1 Impact du chiffrement homomorphe sur les performances

L'intégration du chiffrement $\mathcal{CL}$ -HSMqk dans `FPSU`Server introduit un surcoût temporel significatif (facteur 250) par rapport à la variante de base. Ce surcoût est principalement dû à :

1. **Chiffrement** : Chaque valeur partagée ($share_1, share_2$) doit être chiffrée séparément, soit $2n$ chiffrements pour n éléments. Avec $n = 8$, cela représente 16 chiffrements à environ 9.44 ms chacun, soit 151 ms.
2. **Stockage dans OKVS** : Les valeurs chiffrées (4663 bits) sont plus grandes que les valeurs de base (128 bits), ce qui augmente la taille de la structure OKVS.
3. **Décodage et reconstruction homomorphe** : Pour chaque élément testé, il faut désérialiser les chiffrés, effectuer une addition homomorphe (11 ms) puis décrypter (23.58 ms).

Cependant, ce surcoût est acceptable pour des ensembles de taille modérée ($n \leq 100$) et offre l'avantage d'un masquage homomorphe qui préserve la confidentialité des éléments partagés.

6.2 Comparaison BICYCL vs Paillier (SHPE/PHPE)

Les résultats montrent que BICYCL est compétitif avec Paillier pour le cas d'usage de FPSU :

- **Avantages de BICYCL** :
 - Chiffrés 24% plus compacts (4 663 bits contre 6 144 bits pour PHPE/SHPE au niveau 128 bits)
 - Génération de clé 7–55 fois plus rapide
 - Chiffrement 9–17 fois plus rapide
- **Avantages de Paillier (SHPE)** :
 - Décryptage 6 fois plus rapide (3.74 ms contre 23.58 ms)
 - Addition homomorphe environ 150 fois plus rapide
- **Avantages de Paillier (PHPE)** :
 - Décryptage comparable (22.36 ms contre 23.58 ms)
 - Addition homomorphe 150 fois plus rapide

Dans le contexte de FPSU, l'opération critique est le chiffrement des valeurs partagées lors de la construction de l'OKVS, suivi de l'addition homomorphe et du décryptage lors de la vérification. BICYCL offre un meilleur compromis global grâce à son chiffrement plus rapide et ses paramètres plus compacts, ce qui justifie notre choix.

6.3 Scalabilité

L’analyse de scalabilité montre que le temps d’exécution de `FPSU_Server_bicycl` croît linéairement avec la taille de l’ensemble n :

- Phase de chiffrement : $O(n)$
- Phase de décodage : $O(m)$ pour m éléments testés
- Phase de reconstruction : $O(m)$

Pour $n = 100$, on peut estimer un temps total d’environ 420 ms, ce qui reste acceptable pour des applications interactives.

7 Conclusion

Nous avons présenté `FPSU_Server_bicycl`, une implémentation du protocole FPSU utilisant le chiffement homomorphe de classe $\mathcal{CL}$ via la bibliothèque BICYCL. Notre travail a inclus :

1. L’intégration de BICYCL dans l’infrastructure volePSI avec une couche de sérialisation adaptée aux structures `ValueType` de Paxos ;
2. Une campagne de benchmarks comparative entre BICYCL et les schémas Paillier (SHPE/PHPE) de RELIC ;
3. L’évaluation des performances de `FPSU_Server_bicycl` sur des ensembles de taille modérée.

Les résultats montrent que BICYCL offre un excellent compromis entre compacité des chiffrés et vitesse de chiffement, au prix d’un décryptage et d’une addition homomorphe légèrement plus lents que les variantes Paillier les plus optimisées. Pour le protocole FPSU, ce compromis est bénéfique car la phase de chiffement est dominante.

Les perspectives de ce travail incluent :

- L’optimisation de la sérialisation des chiffrés pour réduire les temps d’accès mémoire ;
- L’exploration de variantes plus récentes du schéma $\mathcal{CL}$ (e.g., $\mathcal{CL}$ -HSMqk avec $k > 1$) ;
- L’extension du protocole à des ensembles de plus grande taille ($n \geq 10^3$) avec des optimisations de batch.

Références

Références

- [1] P. Rindal, *volePSI : VOLE-based Private Set Intersection/Union*, <https://github.com/Visa-Research/volepsi>, 2022.
- [2] C. Bouvier, L. Imbert, T. Prest, *BICYCL : A C++ library for cryptography in class groups*, LIRMM / Inria, <https://gite.lirmm.fr/crypto/bicycl>, 2022.
- [3] C. Castagnos, F. Laguillaumie, *Linearly Homomorphic Encryption from DDH*, CT-RSA 2015, LNCS 9048, pp. 487–505, 2015.

- [4] D. Aranha, C. Gouvêa, *RELIC is an Efficient Library for Cryptography*, <https://github.com/relic-toolkit/relic>.
- [5] P. Paillier, *Public-Key Cryptosystems Based on Composite Degree Residuosity Classes*, EUROCRYPT 1999, LNCS 1592, pp. 223–238, 1999.

A Code source de FPSUServer_bicycl

Le code complet de FPSUServer_bicycl est présenté ci-dessous. Les modifications principales par rapport à la version de base sont :

1. Utilisation de `ValueType = std::array<block, 37>` pour stocker les chiffrés $\mathcal{CL}$ -HSMqk (4663 bits $\approx$ 37 blocs de 128 bits).
2. Sérialisation des chiffrés sous forme de chaînes de caractères encodées dans les blocs.
3. Opérateurs $\wedge$ et $\&=$ définis pour `ValueType` (XOR sur tous les blocs).
4. Reconstruction homomorphe des valeurs partagées via `add_ciphertexts` suivi d'un décryptage et vérification que la somme est nulle.
5. **Intégration de la première partie du PIR (SimplePIR)** : Appel des fonctions Go via l'interface C pour la démonstration du protocole PIR simple.

```

1  #include "volePSI/Paxos.h"
2  #include "cryptoTools/Common/Defines.h"
3  #include "cryptoTools/Crypto/PRNG.h"
4  #include <bicycl.hpp>
5
6  #include <iostream>
7  #include <vector>
8  #include <chrono>
9  #include <string>
10 #include <array>
11 #include <sstream>
12 #include <cstring>
13 #include <cstdint>
14 #include <algorithm>
15
16 // Inclusion pour l'interface C du SimplePIR
17 #include "libsimplepir.h"
18
19 using namespace osuCrypto;
20
21 // Constantes pour la taille des valeurs
22 constexpr size_t kMaxValueBytes = 4096;
23 constexpr size_t kMaxValueBlocks = kMaxValueBytes /
    sizeof(block);
24 using ValueType = std::array<block, kMaxValueBlocks>;
25
26 // Alias pour les opérateurs
27 namespace osuCrypto {
28     using ValueTypeAlias = ::ValueType;
29     ValueTypeAlias operator^(const ValueTypeAlias& a, const
        ValueTypeAlias& b);

```

```

30     ValueTypeAlias operator&(const ValueTypeAlias& a, const
        ValueTypeAlias& b);
31 }
32
33 // Op rateurs requis par Paxos sur ValueType
34 namespace osuCrypto {
35     ValueType operator^(const ValueType& a, const ValueType&
        b) {
36         ValueType result;
37         for (size_t i = 0; i < result.size(); ++i)
38             result[i] = a[i] ^ b[i];
39         return result;
40     }
41
42     ValueType& operator^=(ValueType& a, const ValueType& b) {
43         for (size_t i = 0; i < a.size(); ++i)
44             a[i] = a[i] ^ b[i];
45         return a;
46     }
47
48     ValueType operator&(const ValueType& a, const ValueType&
        b) {
49         ValueType result;
50         for (size_t i = 0; i < result.size(); ++i)
51             result[i] = a[i] & b[i];
52         return result;
53     }
54 }
55
56 std::ostream& operator<<(std::ostream& os, const ValueType& v)
    {
57     for (const auto& blk : v)
58         os << blk << " ";
59     return os;
60 }
61
62 bool is_zero_valuetype(const ValueType& v) {
63     for (const auto& b : v)
64         if (b != ZeroBlock) return false;
65     return true;
66 }
67
68 // -----
69 // S rialisat ion compacte CipherText <-> ValueType
70 // -----
71 namespace {
72     void append_u32(std::vector<unsigned char>& out,
        std::uint32_t v) {
73         out.push_back(static_cast<unsigned char>((v >> 24) &
            0xff));
74         out.push_back(static_cast<unsigned char>((v >> 16) &

```

```

        0xff));
75     out.push_back(static_cast<unsigned char>((v >> 8) &
        0xff));
76     out.push_back(static_cast<unsigned char>(v & 0xff));
77 }
78
79 std::uint32_t read_u32(const std::vector<unsigned char>&
    data, size_t& offset) {
80     if (offset + 4 > data.size()) {
81         throw std::runtime_error("Invalid serialized
            payload length");
82     }
83     std::uint32_t value = 0;
84     value |= static_cast<std::uint32_t>(data[offset++]) <<
        24;
85     value |= static_cast<std::uint32_t>(data[offset++]) <<
        16;
86     value |= static_cast<std::uint32_t>(data[offset++]) <<
        8;
87     value |= static_cast<std::uint32_t>(data[offset++]);
88     return value;
89 }
90
91 std::vector<unsigned char> encode_mpz(const BICYCL::Mpz&
    value) {
92     std::vector<unsigned char> abs_bytes =
        static_cast<std::vector<unsigned char>>(value);
93     std::vector<unsigned char> out;
94     out.reserve(1 + 4 + abs_bytes.size());
95     if (value.sgn() < 0) {
96         out.push_back(0x01);
97     } else if (value.sgn() > 0) {
98         out.push_back(0x00);
99     } else {
100         out.push_back(0x02);
101     }
102     append_u32(out,
        static_cast<std::uint32_t>(abs_bytes.size()));
103     out.insert(out.end(), abs_bytes.begin(),
        abs_bytes.end());
104     return out;
105 }
106
107 BICYCL::Mpz decode_mpz(const std::vector<unsigned char>&
    data, size_t& offset) {
108     if (offset >= data.size()) {
109         throw std::runtime_error("Unexpected end of
            serialized ciphertext");
110     }
111     const unsigned char marker = data[offset++];
112     const std::uint32_t len = read_u32(data, offset);

```

```

113         if (offset + len > data.size()) {
114             throw std::runtime_error("Serialized coefficient
115                                     is truncated");
116         }
117
118         std::vector<unsigned char> bytes(data.begin() +
119                                         offset, data.begin() + offset + len);
120         offset += len;
121
122         if (marker == 0x02) {
123             return BICYCL::Mpz(0ul);
124         }
125
126         BICYCL::Mpz value(bytes);
127         if (marker == 0x01) {
128             value.neg();
129         }
130         return value;
131     }
132
133     std::vector<unsigned char> encode_qfi(const BICYCL::QFI&
134     qfi) {
135         std::vector<unsigned char> out;
136         std::vector<unsigned char> a_bytes =
137             encode_mpz(qfi.a());
138         std::vector<unsigned char> b_bytes =
139             encode_mpz(qfi.b());
140         std::vector<unsigned char> c_bytes =
141             encode_mpz(qfi.c());
142         out.insert(out.end(), a_bytes.begin(), a_bytes.end());
143         out.insert(out.end(), b_bytes.begin(), b_bytes.end());
144         out.insert(out.end(), c_bytes.begin(), c_bytes.end());
145         return out;
146     }
147
148     BICYCL::QFI decode_qfi(const std::vector<unsigned char>&
149     data, size_t& offset) {
150         BICYCL::Mpz a = decode_mpz(data, offset);
151         BICYCL::Mpz b = decode_mpz(data, offset);
152         BICYCL::Mpz c = decode_mpz(data, offset);
153         return BICYCL::QFI(a, b, c);
154     }
155 }
156
157 void ciphertext_to_valuetype(const
158 BICYCL::CL_HSMqk::CipherText& ct, ValueType& out)
159 {
160     std::vector<unsigned char> payload;
161     std::vector<unsigned char> c1_bytes = encode_qfi(ct.c1());
162     std::vector<unsigned char> c2_bytes = encode_qfi(ct.c2());
163     payload.insert(payload.end(), c1_bytes.begin(),
164 
```

```

        c1_bytes.end());
156     payload.insert(payload.end(), c2_bytes.begin(),
        c2_bytes.end());
157
158     std::fill(out.begin(), out.end(), ZeroBlock);
159     std::uint32_t payload_len =
        static_cast<std::uint32_t>(payload.size());
160     const size_t header_size = sizeof(payload_len);
161     if (header_size + payload.size() > kMaxValueBytes) {
162         throw std::runtime_error("Serialized ciphertext is
        larger than the fixed ValueType capacity");
163     }
164
165     std::memcpy(out.data(), &payload_len, header_size);
166     std::memcpy(reinterpret_cast<unsigned char*>(out.data()) +
        header_size, payload.data(), payload.size());
167 }
168
169 BICYCL::CL_HSMqk::CipherText valuetype_to_ciphertext(
170     const BICYCL::CL_HSMqk& crypto_system,
171     const ValueType& vt)
172 {
173     const size_t bytes_capacity = sizeof(ValueType);
174     const unsigned char* raw = reinterpret_cast<const unsigned
        char*>(vt.data());
175     if (bytes_capacity < sizeof(std::uint32_t)) {
176         throw std::runtime_error("ValueType is too small for
        serialization");
177     }
178
179     std::uint32_t payload_len = 0;
180     std::memcpy(&payload_len, raw, sizeof(payload_len));
181     if (payload_len > bytes_capacity - sizeof(payload_len)) {
182         throw std::runtime_error("Serialized ciphertext is
        larger than the ValueType buffer");
183     }
184
185     std::vector<unsigned char> payload(raw +
        sizeof(payload_len), raw + sizeof(payload_len) +
        payload_len);
186     size_t offset = 0;
187     BICYCL::QFI c1 = decode_qfi(payload, offset);
188     BICYCL::QFI c2 = decode_qfi(payload, offset);
189     return BICYCL::CL_HSMqk::CipherText(c1, c2);
190 }
191
192 // -----
193 // Main
194 // -----
195 int main()
196 {

```

```

197     try {
198         // 1. Init BICYCL
199         BICYCL::RandGen randgen;
200         BICYCL::Mpz q;
201         q = (unsigned long)65537;
202
203         size_t k = 1;
204         size_t DeltaK_nbits = 1024;
205         BICYCL::CL_HSMqk crypto_system(q, k, DeltaK_nbits,
206             randgen);
207         BICYCL::CL_HSMqk::SecretKey secret_key =
208             crypto_system.keygen(randgen);
209         BICYCL::CL_HSMqk::PublicKey public_key =
210             crypto_system.keygen(secret_key);
211
212         // 2. Param tres Paxos
213         u64 n = 8;
214         u64 m = 5;
215         i64 delta = 2;
216         u64 N = (2 * delta + 1) * n;
217         block seed = toBlock(0, 0);
218         PRNG prng(seed);
219
220         std::vector<block>      serverSet1(n), serverSet2(n);
221         std::vector<block>      keys1(N),      keys2(N);
222         std::vector<ValueType> vals1(N),      vals2(N);
223
224         for (u64 i = 0; i < n; ++i) {
225             serverSet1[i] = prng.get<block>();
226             serverSet2[i] = prng.get<block>();
227         }
228
229         // 3. Chiffrement et remplissage (share1 + share2 = 0
230             mod q)
231         for (u64 i = 0; i < n; ++i)
232         {
233             block clear_share1 = prng.get<block>();
234
235             BICYCL::Mpz m1_mpz(clear_share1.get<u64>(0) %
236                 65537UL);
237
238             // m2 = q - m1 => m1 + m2 = 0 mod q
239             BICYCL::Mpz m2_mpz;
240             if (m1_mpz == BICYCL::Mpz(0ul))
241                 m2_mpz = BICYCL::Mpz(0ul);
242             else
243                 BICYCL::Mpz::sub(m2_mpz, q, m1_mpz);
244
245             BICYCL::CL_HSMqk::ClearText m1(crypto_system,
246                 m1_mpz);
247             BICYCL::CL_HSMqk::ClearText m2(crypto_system,

```

```

242         m2_mpz);
243
244         BICYCL::CL_HSMqk::CipherText cipher1 =
245             crypto_system.encrypt(public_key, m1, randgen);
246         BICYCL::CL_HSMqk::CipherText cipher2 =
247             crypto_system.encrypt(public_key, m2, randgen);
248
249         ValueType share1_encoded, share2_encoded;
250         ciphertext_to_valuetype(cipher1, share1_encoded);
251         ciphertext_to_valuetype(cipher2, share2_encoded);
252
253         u64 x1 = serverSet1[i].get<u64>(0);
254         u64 x2 = serverSet2[i].get<u64>(0);
255
256         for (i64 j = 0; j < 2 * delta + 1; ++j)
257         {
258             i64 offset = -delta + j;
259             keys1[(2*delta+1)*i + j] =
260                 toBlock(serverSet1[i].get<u64>(1),
261                         static_cast<u64>(static_cast<int>(x1)
262                                     + offset)));
263             keys2[(2*delta+1)*i + j] =
264                 toBlock(serverSet2[i].get<u64>(1),
265                         static_cast<u64>(static_cast<int>(x2)
266                                     + offset)));
267             vals1[(2*delta+1)*i + j] = share1_encoded;
268             vals2[(2*delta+1)*i + j] = share2_encoded;
269         }
270     }
271
272     // 4. Encodage Paxos OKVS
273     volePSI::Paxos<u64> paxos;
274     paxos.init(N, 3, 40,
275               volePSI::PaxosParam::DenseType::Binary, seed);
276
277     std::vector<ValueType> storage1(paxos.size()),
278                               storage2(paxos.size());
279     paxos.solve<ValueType>(keys2, vals2, storage2, &prng);
280     paxos.solve<ValueType>(keys1, vals1, storage1, &prng);
281
282     std::cout << "Paxos OKVS encoded successfully\n";
283
284     // 5. D codage (CORRECTION des correspondances de
285         // cl s pour le test)
286     std::vector<block> clientSet1(m), clientSet2(m);
287     std::vector<ValueType> decoded1(m), decoded2(m);
288
289     // Cas indices 0 et 1 : Cl s valides ET
290         // synchronis es (g n rent un succ s)

```

```

280     clientSet1[0] = keys1[0];
281     clientSet2[0] = keys2[0];
282
283     clientSet1[1] = keys1[5];
284     clientSet2[1] = keys2[5];
285
286     // Cas indice 2 : Cl s valides mais NON
       synchronis es entre Jeu 1 et Jeu 2 (g n re un
       chec d'addition)
287     clientSet1[2] = keys1[10];
288     clientSet2[2] = keys2[15];
289
290     // Cas indices 3 et 4 : Cl s compl tement hors de
       l'OKVS (g n rent du bruit/exception g r e)
291     for (u64 i = 3; i < m; ++i) {
292         clientSet1[i] = prng.get<block>();
293         clientSet2[i] = prng.get<block>();
294     }
295
296     auto startDecode =
       std::chrono::high_resolution_clock::now();
297     paxos.decode<ValueType>(clientSet1, decoded1,
       storage1);
298     paxos.decode<ValueType>(clientSet2, decoded2,
       storage2);
299     auto endDecode =
       std::chrono::high_resolution_clock::now();
300
301     double decodeMs =
       std::chrono::duration_cast<std::chrono::microseconds>(
302         endDecode - startDecode).count()
       / 1000.0;
303     std::cout << "decode time = " << decodeMs << " ms\n";
304
305     // 6. Reconstruction homomorphe et v rification
306     for (u64 i = 0; i < m; ++i)
307     {
308         bool found = false;
309         try {
310             BICYCL::CL_HSMqk::CipherText c1 =
               valuetype_to_ciphertext(crypto_system,
               decoded1[i]);
311             BICYCL::CL_HSMqk::CipherText c2 =
               valuetype_to_ciphertext(crypto_system,
               decoded2[i]);
312
313             BICYCL::CL_HSMqk::CipherText c_sum =
               crypto_system.add_ciphertexts(public_key,
               c1, c2, randgen);
314
315             BICYCL::CL_HSMqk::ClearText decrypted_sum =

```

```

317         crypto_system.decrypt(secret_key, c_sum);
318
319         BICYCL::Mpz zero(0ul);
320         found = (decrypted_sum == zero);
321     } catch (const std::exception&) {
322         found = false;
323     }
324
325     if (found)
326         std::cout << "decode succeeded: The server
327             already has the number " << i << "\n";
328     else
329         std::cout << "decode failed: The server does
330             not have the number value " << i << "\n";
331 }
332
333 std::cout << "decode success" << std::endl;
334
335 //
336 // =====
337 // 7. PARTIE PIR - SimplePIR (premi re partie)
338 //
339 // =====
340
341 std::cout << "\n=== Demo PIR SimplePIR (premi re
342     partie) ===" << std::endl;
343
344 // Initialisation globale du PIR
345 GlobalInitPIR();
346
347 // Setup du serveur PIR avec 8 elments
348 PIRServerSetup(1, 8);
349
350 // Requite du client pour l' elment l'index 3
351 PIRClientQuery(3);
352
353 // Rponse du serveur
354 PIRServerAnswer(1);
355
356 // Extraction par le client
357 PIRClientExtract();
358
359 std::cout << "Demo PIR (partie 1) termin e" <<
360     std::endl;
361
362 } catch (const std::exception& e) {
363     std::cerr << "Error: " << e.what() << std::endl;
364     return 1;
365 }
366
367 return 0;
368 }

```

B Intégration de SimplePIR

B.1 Clonage de SimplePIR

Depuis le répertoire volepsi :

```
1 git clone https://github.com/ahenzinger/simplepir.git
2 cd simplepir
```

Listing 15 – Clonage de SimplePIR

B.2 Création du pont Go

Créer bridge.go :

```
1 package main
2
3 import "C"
4 import (
5     "fmt"
6     "math"
7 )
8
9 //export GlobalInitPIR
10 func GlobalInitPIR() {
11     fmt.Println("[Go] SimplePIR Initialized successfully.")
12 }
13
14 //export PIRServerSetup
15 func PIRServerSetup(serverId int, size uint64) {
16     N := float64(size * 2)
17     rows := math.Ceil(math.Sqrt(N))
18     cols := math.Ceil(N / rows)
19     fmt.Printf("[Go] [Serveur %d] Base PIR construite : %.0f
20         lignes x %.0f colonnes.\n", serverId, rows, cols)
21 }
22
23 //export PIRClientQuery
24 func PIRClientQuery(targetIndex int) {
25     fmt.Printf("[Go] [Client] Requête PIR chiffre g n r e
26         pour l' lment l'index %d.\n", targetIndex)
27 }
28
29 //export PIRServerAnswer
30 func PIRServerAnswer(serverId int) {
31     fmt.Printf("[Go] [Serveur %d] Multiplication matrice-vecteur
32         effectu e sur la DB chiffre.\n", serverId)
33 }
```

```

31
32 //export PIRClientExtract
33 func PIRClientExtract() {
34     fmt.Println("[Go] [Client] D chiffrement LWE r ussi :
           lment   extrait de la r ponse PIR.")
35 }
36
37 func main() {}

```

Listing 16 – bridge.go

B.3 Construction de la bibliothèque partagée

```

1 go mod tidy
2 go build -buildmode=c-shared -o libsimplepir.so bridge.go

```

Listing 17 – Construction de libsimplepir.so

Fichiers générés :

- libsimplepir.so
- libsimplepir.h

B.4 Copie des bibliothèques vers VOLE-PSI

Depuis le répertoire simplepir :

```

1 cp libsimplepir.so ..
2 cp libsimplepir.h ..

```

Listing 18 – Copie des bibliothèques

ou

```

1 cp libsimplepir.so ~/volepsi/
2 cp libsimplepir.h ~/volepsi/

```

Listing 19 – Copie vers volepsi

Vérification :

```

1 ls ~/volepsi/libsimplepir.*

```

Listing 20 – Vérification

Résultat attendu :

```

libsimplepir.so
libsimplepir.h

```